

Reviewer Pack — Minimal Geometric Measure of Boolean Functions and Communica...

Entry 15426a361009 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 20:08:02 UTC

Minimal Geometric Measure of Boolean Functions and Communication Complexity Rank

Entry ID: 15426a361009 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 20:08:02 UTC

1. Conjecture

Field A (mathematical branch): Geometric Measure Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For all boolean functions $f: \{0, \dots, N-1\}^m \rightarrow \{0, \dots, d-1\}$, the geometric measure of f , defined as the volume of the unit sphere in the m -dimensional real vector space spanned by the Fourier coefficients of f , is linearly correlated with the communication complexity rank of the corresponding matrix representation M_f , such that $\text{GeoMeasure}(f) = \Theta(\text{Rank}(M_f))$.

Rationale (proposer's reasoning):

Geometric Measure Theory provides a way to quantify the 'size' of geometric objects in a Euclidean space. Applying this to the Fourier coefficients of boolean functions could potentially reveal hidden structural properties related to communication complexity rank, which measures the minimum size of a matrix that can represent the function. This correlation might expose new insights into the relationship between classical and quantum computation.

Taxonomy category: geometric_measure_theory_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c553f5a183489852

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given boolean function f , if the Pearson's correlation coefficient r between $\text{GeoMeasure}(f)$ and $\text{Rank}(M_f)$ is greater than or equal to 0.7 over 30 random seeds, the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:(Geometric Measure Theory AND Communication Complexity Rank) AND boolean functions - title:(volume of unit sphere AND Fourier coefficients) AND communication complexity - arxiv:math.PR OR cs.DC AND (geometric measure theory AND matrix rank)

Top relevant hits considered: - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/2406.18700v4] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/0704.2351v2] Parallel computation of the rank of large sparse matrices from algebraic K-theory - [http://arxiv.org/abs/math-ph/0301018v3] Conformal restriction, highest-weight representations and SLE - [http://arxiv.org/abs/1010.0476v3] Interference Alignment as a Rank Constrained Rank Minimization

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.randint(0, 1) for _ in range(2**n)]

    def fourier_coefficients(f):
        n = int(math.log2(len(f)))
        coeffs = [0] * (2*n + 1)
        for i in range(len(f)):
            for j in range(2*n + 1):
                angle = 2 * math.pi * j * i / len(f)
                coeffs[j] += f[i] * math.cos(angle) / len(f)
        return coeffs

    def geometric_measure(coeffs):
        sum_of_squares = sum(x**2 for x in coeffs)
```

```

    return math.sqrt(sum_of_squares)

def communication_complexity_rank(M):
    n, m = len(M), len(M[0])
    rank = 0
    for i in range(n):
        if M[i][i] != 0:
            rank += 1
            for j in range(i+1, n):
                M[j][i] /= M[i][i]
                for k in range(m):
                    M[j][k] -= M[j][i] * M[i][k]
    return rank

def matrix_representation(f):
    n = int(math.log2(len(f)))
    M = [[0] * (n+1) for _ in range(n+1)]
    for i in range(2**n):
        binary_rep = format(i, f'0{n}b')
        for j in range(n):
            if binary_rep[j] == '1':
                M[j][j+1] += 1
            else:
                M[j+1][j] += 1
    return M

n_max = 40
instances_tested = 30
metric_values = []
conjecture_holds = True
counterexample = ""

for n in range(5, n_max + 1):
    f = generate_boolean_function(n)
    coeffs = fourier_coefficients(f)
    geo_measure = geometric_measure(coeffs)
    M = matrix_representation(f)
    rank = communication_complexity_rank(M)

    if geo_measure == 0 or rank == 0:
        continue

    metric_values.append(geo_measure * rank)

if len(metric_values) < instances_tested:
    conjecture_holds = False
    counterexample = "not_enough_instances"

return {
    "metric_name": "GeoMeasure * Rank",
    "metric_value": sum(metric_values) / len(metric_values),
    "instances_tested": len(metric_values),
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"not_enough_support\" first_failing_seed={seeds[0]}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition was not unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15884
2	propose	ollama_remote	glm4:latest	0	0	12823
3	preregistration	ollama_remote	glm4:latest	0	0	9353
4	novelty	ollama_remote	glm4:latest	0	0	14083
5	novelty	ollama_remote	glm4:latest	0	0	9464

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15636
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13849
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10551
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10479
10	judge	ollama_remote	glm4:latest	0	0	44979

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 157103 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/15426a361009.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/15426a361009.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/15426a361009.tar.gz* (if generated)